

Mundo dos Blocos de Tamanho Variável

Representação do Conhecimento e Raciocínio Automatizado

Fundamentos de Inteligência Artificial – Prof. Edjard Mota

Equipe 5

Fernando Naum dos Santos Batista, Gabriel Lucas Araújo Matos,
Guilherme Paiva Orlando, Luis Felipe Ferreira Cavalcante,
Yuri Garcia e Silva

2 de outubro de 2026

Sumário

1	Introdução ao Problema	3
1.1	O domínio	3
1.2	Objetivo do trabalho	3
1.3	As três situações	4
1.4	Organização do documento	4
2	Descrição Formal do Mundo dos Blocos de Tamanho Variável	4
2.1	Representação em Lógica de Primeira Ordem	4
2.1.1	Vocabulário	4
2.1.2	Leis do domínio	5
2.1.3	Ação $move(b, y, p)$	5
2.1.4	Estados das três situações	6
2.2	Efeitos das ações e tabela de adds/deletes	7
2.2.1	Ações associadas aos elementos da representação	7
2.2.2	Relação entre os símbolos e a ação move	7
2.2.3	Efeitos ADD e DELETE da ação	7
2.2.4	Predicados derivados	8
2.2.5	Estabilidade após o movimento	8
2.2.6	Aplicações	8
2.3	Ordem Parcial	9
2.3.1	Plano de ordem parcial	9
2.3.2	Execução manual: planos do item 3 como ordem parcial	10
2.3.3	Ordem parcial entre metas	11
2.4	Execução Manual dos Cenários	11
2.4.1	Situação 1: $S_0 \rightarrow S_{f3}$	12
2.4.2	Situação 1 (extra): $S_0 \rightarrow S_{f4}$	13
2.4.3	Situação 2: $S_0 \rightarrow S_5$	14
2.4.4	Situação 3: $S_0 \rightarrow S_7$	15

3	Codificação CNF	17
3.1	Variáveis proposicionais	17
3.2	Grupos de cláusulas	17
3.3	Estabilidade em cláusulas	19
3.4	Como ADD e DELETE aparecem na codificação	19
3.5	A relação <i>on</i> é derivada	19
3.6	Horizonte e plano mínimo	19
4	Exemplos dos 3 Cenários Codificados para CNF	19
4.1	Situação 1	20
4.2	Situação 2	20
4.3	Situação 3	20
4.4	Cenário com ordem parcial entre metas	20
5	Mapeamento: Descrição Formal $\rightarrow$ Código	20
5.1	Diferenças em relação ao manual do professor	21
6	Execução Passo a Passo	22
6.1	Uma situação, passo a passo	22
6.2	Plano mínimo automático	22
6.3	Horizontes testados	22
6.4	Com ordem parcial entre metas	23
6.5	Arquivos entregues	23
7	Interpretação da Saída do SAT Solver	23
7.1	O minisat não conhece nomes simbólicos	23
7.2	Da saída numérica ao plano em português	23
7.3	Exemplo: Situação 3	24
7.4	Comparação com os planos manuais	24
7.5	Com ordem parcial entre metas	25

1 Introdução ao Problema

O Mundo dos Blocos é um domínio clássico de planejamento em Inteligência Artificial. Dado um estado inicial e um estado objetivo, o problema é encontrar uma sequência de ações que leve de um ao outro. Na versão clássica, todos os blocos têm o mesmo tamanho e cada bloco ocupa exatamente uma posição. Neste trabalho estendemos o domínio para blocos de *comprimentos diferentes*, dispostos sobre uma mesa com pontos discretos e empilháveis em níveis.

1.1 O domínio

Temos quatro blocos a , b , c e d , com comprimentos $\ell(a) = \ell(b) = 1$, $\ell(c) = 2$ e $\ell(d) = 3$. A mesa é um segmento com os pontos inteiros de 0 a 6, o que dá seis *slots* unitários, $s_i = [i, i + 1]$ para $i \in \{0, \dots, 5\}$. Um bloco começa em um ponto p e cobre $\ell(b)$ slots consecutivos. Os blocos podem ficar sobre a mesa (nível 0) ou sobre outros blocos (níveis acima).

Essa extensão traz quatro características que o caso simples não tem:

- **Tamanhos diferentes:** um bloco pode cobrir mais de um slot e se apoiar em mais de um bloco.
- **Espaços vazios:** o bloco do nível de cima pode ficar em ponte sobre dois blocos de baixo, deixando um slot vazio entre eles.
- **Movimentos laterais:** além de subir e descer, um bloco pode deslizar na horizontal, então a posição em que ele é colocado importa.
- **Equilíbrio:** um bloco maior sobre um menor só é aceito se tiver apoio suficiente por baixo. A regra adotada exige apoio em pelo menos $\lceil \ell(b)/2 \rceil$ dos slots que o bloco cobre.

1.2 Objetivo do trabalho

O objetivo é propor uma representação do conhecimento para esse domínio e usá-la em raciocínio automatizado para gerar planos. O trabalho segue as etapas abaixo, na ordem em que o enunciado pede.

1. Representar o domínio em Lógica de Primeira Ordem (LPO), com os predicados, as leis do domínio e a ação *move*.
2. Indicar, para cada elemento da representação, o que a ação adiciona ou remove (adds e deletes).
3. Usar a representação para obter, manualmente, os planos de cada cenário.
4. Considerar ordem parcial entre as metas, na forma $A \xrightarrow{P} B$.
5. Somente depois, codificar a solução de LPO para lógica proposicional (CNF), gerar os planos com um SAT solver e compará-los com os planos manuais.

1.3 As três situações

O enunciado define três situações sobre o mesmo conjunto de blocos. Na Situação 1, parte-se do estado inicial S_0 e o plano deve chegar a um dos estados objetivo S_{f1} , S_{f2} , S_{f3} ou S_{f4} . Na Situação 2, o plano vai de S_0 até S_5 . Na Situação 3, o plano vai de S_0 até S_7 . Em todas elas a única ação disponível é $move(b, y, p)$, que move o bloco b para cima de y (outro bloco ou a mesa T) começando no ponto p .

1.4 Organização do documento

A Seção 2 apresenta a descrição formal do domínio (itens 1, 2 e 4) e a execução manual das três situações (item 3). As Seções 3 a 7 tratam da codificação em CNF e do uso do SAT solver (item 5): as variáveis e cláusulas, os três cenários codificados, o mapeamento entre a descrição formal e o código, a execução passo a passo e a interpretação da saída do solver.

2 Descrição Formal do Mundo dos Blocos de Tamanho Variável

2.1 Representação em Lógica de Primeira Ordem

2.1.1 Vocabulário

Constantes. Blocos: a, b, c, d . Mesa: T . Instantes: t . Pontos $p \in \{0, \dots, 6\}$. O slot s_i é o intervalo $[i, i + 1]$, com $i \in \{0, \dots, 5\}$. Níveis $l \in \{0, 1, 2\}$. O nível 0 é a mesa. O nível 2 é o maior que aparece nas três situações.

Função de tamanho.

$$\ell(a) = 1 \quad \ell(b) = 1 \quad \ell(c) = 2 \quad \ell(d) = 3$$

Predicados primitivos.

Predicado	Significado
$at(b, p, t)$	O bloco b começa no ponto p no instante t . Posições válidas: $p \in \{0, \dots, 6 - \ell(b)\}$.
$lev(b, l, t)$	O bloco b está no nível l no instante t .

Predicados derivados (axiomas de definição).

$$\begin{aligned}
cov(b, s, l, t) &\leftrightarrow \exists p. at(b, p, t) \wedge lev(b, l, t) \wedge p \leq s < p + \ell(b) \\
span(b, p) &= \{s \in \mathbb{N} : p \leq s < p + \ell(b)\} = [p, p + \ell(b)) \\
overlap(b, p, y, q) &\leftrightarrow span(b, p) \cap span(y, q) \neq \emptyset \quad (\text{isto é, } p < q + \ell(y) \wedge q < p + \ell(b)) \\
free(s, l, b, t) &\leftrightarrow \neg \exists b' \neq b. cov(b', s, l, t) \\
clr(b, t) &\leftrightarrow \neg \exists b' \neq b, s, l. lev(b, l, t) \wedge cov(b, s, l, t) \wedge cov(b', s, l + 1, t) \\
on(b, T, t) &\leftrightarrow lev(b, 0, t) \\
on(b, y, t) &\leftrightarrow \exists s, l. cov(b, s, l, t) \wedge cov(y, s, l - 1, t) \\
stable(b, p, l, t) &\leftrightarrow l = 0 \vee \# \{s \in [p, p + \ell(b)) : \exists b' \neq b. cov(b', s, l - 1, t)\} \geq \lceil \ell(b)/2 \rceil
\end{aligned}$$

Cada característica pedida no enunciado tem um elemento correspondente:

Característica	Elemento da LPO
Tamanhos diferentes	ℓ e cov (os slots cobertos por cada bloco)
Espaços vazios	$free$, por slot e por nível
Equilíbrio (bloco de cima maior que o de baixo)	$stable$
Apoio em dois blocos (ponte)	$on(b, y_1) \wedge on(b, y_2)$, derivado de cov
Dois blocos lado a lado sobre o mesmo apoio	$free$ por slot

2.1.2 Leis do domínio

Valem em todo estado, para todo instante t .

(U1) $\forall b, t. \exists! p. at(b, p, t)$	unicidade de posição
(U2) $\forall b, t. \exists! l. lev(b, l, t)$	unicidade de nível
(L) $at(b, p, t) \rightarrow 0 \leq p \leq 6 - \ell(b)$	limites da mesa
(E) $\neg \exists b \neq b', s, l. cov(b, s, l, t) \wedge cov(b', s, l, t)$	exclusão horizontal
(S) $at(b, p, t) \wedge lev(b, l, t) \rightarrow stable(b, p, l, t)$	estabilidade

2.1.3 Ação $move(b, y, p)$

Significado: mover o bloco b para cima de y (bloco ou mesa T), começando no ponto p . O nível-alvo é $L = 0$ se $y = T$, e $L = l_y + 1$ caso contrário, onde $lev(y, l_y, t)$. A ação é possível em t quando:

$$\begin{aligned}
poss(move(b, y, p), t) \leftrightarrow & clr(b, t) \\
& \wedge y \neq b \wedge 0 \leq p \leq 6 - \ell(b) \wedge L \leq 2 \\
& \wedge (y = T \vee \exists q. at(y, q, t) \wedge overlap(b, p, y, q)) \\
& \wedge \forall s \in [p, p + \ell(b)). free(s, L, b, t) \\
& \wedge stable(b, p, L, t) \\
& \wedge \neg(at(b, p, t) \wedge lev(b, L, t))
\end{aligned}$$

Subir, descer e deslizar lateralmente são o mesmo operador: o nível-alvo L e a posição p decidem qual dos três acontece.

Notação. $move(b, y, p)$ é o termo-ação e o instante t entra apenas em $poss(move(b, y, p), t)$. No manual e na codificação CNF o instante aparece como quarto argumento, $move(b, y, p, t)$ e $mv(b, y, p, t)$; as duas formas são equivalentes. Em $span$, $\mathbb{N}$ é o conjunto dos inteiros não negativos (índices de slot).

Desvio em relação ao manual. O manual exige $clr(y)$, o topo inteiro do bloco de destino livre. Aqui essa exigência foi trocada por $free$ por slot, que é o que a pré-condição 5 do manual já pede: nenhum outro bloco ocupa os slots que b vai cobrir no nível-alvo. Os próprios slides precisam disso. Nas Situações 2 e 3, a transição $S_4 \rightarrow S_5$ coloca o b sobre o c ao lado do a , que já está lá. Com $clr(y)$, essas transições são impossíveis. Em S_{f1} e S_{f2} da Situação 1, a e b também ficam lado a lado sobre o mesmo bloco.

2.1.4 Estados das três situações

Cada estado é uma conjunção, no instante correspondente, de $at(b, p) \wedge lev(b, l)$ para os quatro blocos. Escrevemos cada bloco como $b(p, l)$. Por exemplo, S_0 da Situação 1 expande para:

$$at(c, 0, t) \wedge lev(c, 0, t) \wedge at(a, 3, t) \wedge lev(a, 0, t) \wedge at(b, 5, t) \wedge lev(b, 0, t) \wedge at(d, 3, t) \wedge lev(d, 1, t)$$

Situação 1.

Estado	Posições $b(p, l)$
S_0	$c(0, 0) \ a(3, 0) \ b(5, 0) \ d(3, 1)$
S_{f1}	$d(3, 0) \ a(4, 1) \ b(5, 1) \ c(4, 2)$
S_{f2}	$d(3, 0) \ c(4, 1) \ a(4, 2) \ b(5, 2)$
S_{f3}	$c(0, 0) \ a(2, 0) \ d(0, 1) \ b(5, 0)$
S_{f4}	$c(0, 0) \ a(0, 1) \ d(2, 0) \ b(5, 0)$

Situação 2.

Estado	Posições $b(p, l)$
S_0	$c(0, 0) \ a(0, 1) \ b(1, 1) \ d(3, 0)$
S_1	$c(0, 0) \ a(0, 1) \ b(2, 0) \ d(3, 0)$
S_2	$c(0, 0) \ b(2, 0) \ a(2, 1) \ d(3, 0)$
S_3	$b(2, 0) \ a(2, 1) \ d(3, 0) \ c(4, 1)$
S_4	$b(2, 0) \ d(3, 0) \ c(4, 1) \ a(4, 2)$
S_5	$d(3, 0) \ c(4, 1) \ a(4, 2) \ b(5, 2)$

Situação 3.

Estado	Posições $b(p, l)$
S_0	$c(0, 0) \ a(3, 0) \ b(5, 0) \ d(3, 1)$
S_1	$c(0, 0) \ a(3, 0) \ b(5, 0) \ d(0, 1)$
S_2	$c(0, 0) \ d(0, 1) \ b(5, 0) \ a(5, 1)$
S_3	$c(0, 0) \ d(2, 0) \ b(5, 0) \ a(5, 1)$
S_4	$c(0, 0) \ d(2, 0) \ b(5, 0) \ a(0, 1)$
S_5	$c(0, 0) \ d(2, 0) \ a(0, 1) \ b(1, 1)$
S_6	igual a S_5
S_7	$c(0, 0) \ d(3, 0) \ a(0, 1) \ b(1, 1)$

Verificação e observações.

- Todos os estados acima satisfazem as leis (U1), (U2), (L), (E) e (S). Em S_0 da Situação 1, por exemplo, o d cobre os slots 3, 4 e 5 no nível 1 e tem apoio nos slots 3 (a) e 5 (b): $2 \geq \lceil 3/2 \rceil$. Daí vem $on(d, a) \wedge on(d, b)$, com o slot 4 vazio embaixo (ponte).
- No slide da Situação 3, S_5 e S_6 estão desenhados iguais, e isso é intencional no enunciado. S_6 é o mesmo estado de S_5 (repetição, sem ação entre eles), e o plano segue de S_5 direto para S_7 .

- O S_0 da Situação 3 é o mesmo S_0 da Situação 1.

2.2 Efeitos das ações e tabela de adds/deletes

2.2.1 Ações associadas aos elementos da representação

Os predicados at e lev são os únicos elementos que são alterados pelo $move$. Já os demais elementos são estáticos, como o $\ell(b)$, ou são derivados: cov , $free$, clr , on e $stable$. Portanto seus valores são obtidos a partir do at , lev e $\ell(b)$.

2.2.2 Relação entre os símbolos e a ação move

Símbolo	ADD	DELETE	Uso
$\ell(b)$	–	–	Propriedade estática usada para limites, cobertura e estabilidade.
$at(b, p, t)$	Nova posição de b	Posição anterior de b	Posição do bloco.
$lev(b, l, t)$	Novo nível de b	Nível anterior de b	Nível do bloco.
$cov(b, s, l, t)$	–	–	Derivado de at , lev e $\ell(b)$.
$free(s, l, b, t)$	–	–	Pré-condição do destino.
$clr(b, t)$	–	–	Pré-condição para mover b .
$on(b, y, t)$	–	–	Derivado de cov .
$stable(b, p, l, t)$	–	–	Testado para garantir a estabilidade do destino.
$span(b, p)$	–	–	Derivado de p e $\ell(b)$, representando os espaços ocupados pelo bloco.
$overlap(b, p, y, q)$	–	–	Testado em $poss(move(b, y, p), t)$ para verificar sobreposição com o apoio.

A ação é $move(b, y, p)$, onde b é o bloco a ser movimentado, y o destino (bloco ou mesa) e p é a nova posição de b . As condições de possibilidade dessa ação já são definidas por $poss(move(b, y, p), t)$, incluindo clr , $free$, limites de posições e $stable$.

2.2.3 Efeitos ADD e DELETE da ação

Se a ação $move(b, y, p)$ é executada no instante t , o bloco b passa a ocupar uma nova posição e novo nível no instante $t + 1$.

ADD. A nova posição e o novo nível são adicionados:

$$at(b, p, t + 1) \quad lev(b, L, t + 1)$$

Onde $L = 0$ se o destino for a mesa T e, se y for outro bloco com $lev(y, l_y, t)$, então $L = l_y + 1$.

DELETE. A posição anterior do bloco b deixa de valer, desde que seja diferente da nova. Se antes da ação:

$$at(b, p_0, t)$$

e $p_0 \neq p$, então após o movimento:

$$\neg at(b, p_0, t + 1)$$

Se $p_0 = p$ (o bloco apenas sobe ou desce no mesmo ponto), o at não é apagado: $at(b, p, t+1)$ já vale pelo ADD, e apagá-lo o contradiria.

Da mesma forma, se o nível anterior for:

$$lev(b, l_0, t)$$

e $l_0 \neq L$, então:

$$\neg lev(b, l_0, t + 1)$$

Se $l_0 = L$ (o bloco desliza lateralmente no mesmo nível), o lev não é apagado.

Portanto, a ação *move* remove diretamente a posição e o nível antigos, quando mudam, e adiciona a nova posição e o novo nível.

2.2.4 Predicados derivados

Os predicados derivados são: *cov*, *free*, *clr*, *on*, *stable*. Após *at* e *lev* serem alterados, esses predicados são recalculados pelas definições apresentadas no item 1. Por exemplo, *on* é derivado a partir da posição, nível e sobreposição dos blocos, enquanto *clr* depende da existência ou não de blocos ocupando posições acima. Isso significa que não é necessário escrever: $\neg clr(y, t + 1)$.

2.2.5 Estabilidade após o movimento

A estabilidade do bloco movimentado deve ser verificada antes que o movimento seja permitido: $stable(b, p, l, t)$. Assim, o novo estado só pode ser obtido se o bloco possuir suporte suficiente na posição e nível de destino.

A estabilidade dos demais blocos não precisa ser verificada novamente porque $clr(b, t)$ é uma pré-condição para mover b . Pela definição de *clr*, isso garante que não existe nenhum bloco ocupando o nível imediatamente acima de b e apoiado sobre sua região. Portanto, ao retirar b de sua posição atual, nenhum outro bloco perde suporte por causa dessa remoção. Assim, basta verificar a estabilidade do próprio bloco movimentado em sua nova posição.

2.2.6 Aplicações

As regras acima são gerais e se aplicam às Situações 1, 2 e 3. Em todos os casos, permanecem os mesmos:

- os efeitos ADD sobre *at* e *lev*;
- os efeitos DELETE da posição e do nível anteriores do bloco movimentado;
- as pré-condições da ação *move*;
- os predicados derivados *cov*, *free*, *clr*, *on* e *stable*;
- os axiomas de persistência dos blocos não movimentados.

Persistência. Para qualquer bloco $b' \neq b$, se a ação executada em t move apenas o bloco b , então b' mantém sua posição e seu nível em $t + 1$:

$$b' \neq b \rightarrow (at(b', p', t) \leftrightarrow at(b', p', t + 1))$$

$$b' \neq b \rightarrow (lev(b', l', t) \leftrightarrow lev(b', l', t + 1))$$

Assim, a ação $move(b, y, p)$ altera diretamente apenas o bloco b . Os demais permanecem inalterados.

Apenas o que muda entre as situações são:

- os estados iniciais;
- os estados objetivos;
- as instâncias da ação $move$.

2.3 Ordem Parcial

Um plano linear $A_1, A_2, \dots, A_k$ fixa uma ordem total entre as ações. Um *plano de ordem parcial* fixa apenas as ordens que são necessárias, e deixa livre a ordem das ações independentes. Esta seção formaliza isso para o mundo dos blocos de tamanho variável (itens 1 e 2) e depois trata a ordem parcial entre as *metas*, que é o que entra na codificação em CNF.

2.3.1 Plano de ordem parcial

Um plano de ordem parcial é uma tripla $\langle \mathcal{A}, \prec, \mathcal{L} \rangle$:

- $\mathcal{A}$ é um conjunto de passos, cada um uma instância $move(b, y, p)$;
- $\prec$ é uma relação de ordem entre passos: irreflexiva e transitiva, logo sem ciclos. $A_i \prec A_j$ significa que A_i é executado antes de A_j ;
- $\mathcal{L}$ é um conjunto de *elos causais* $A_i \xrightarrow{\varphi} A_j$.

Fatos. O estado é dado por at e lev , e as pré-condições de $move$ são derivadas. Os fatos que ligam uma ação à outra são sobre slots:

$$livre(s, l) \equiv free(s, l, \cdot, t) = \neg \exists b'. cov(b', s, l, t) \quad ocupado(s, l) \equiv \neg livre(s, l)$$

Pré-condições e efeitos em termos de fatos. Para $move(b, y, p)$, com b antes em (p_0, l_0) e nível-alvo L :

Pré-condição	Fatos exigidos
(P1) $clr(b, t)$	$livre(s, l_0 + 1)$ para todo $s \in span(b, p_0)$, porque $clr(b, t) \leftrightarrow \forall s \in span(b, p_0). free(s, l_0 + 1, b, t)$
(P4) destino livre	$livre(s, L)$ para todo $s \in span(b, p)$ (exceto os slots que o próprio b já ocupa nesse nível)
(P5) estabilidade	$ocupado(s, L - 1)$ para os slots de $span(b, p)$ que servem de apoio
Efeito	Produz $livre(s, l_0)$ para $s \in span(b, p_0)$ e $ocupado(s, L)$ para $s \in span(b, p)$; desfaz os fatos contrários

Elo causal. $A_i \xrightarrow{\varphi} A_j$ significa que A_i produz o fato φ , A_j exige φ como pré-condição e φ vale em todo instante entre os dois. O elo impõe $A_i \prec A_j$.

Ameaça e resolução. Um passo A_k *ameaça* o elo $A_i \xrightarrow{\varphi} A_j$ se desfaz φ e pode ser executado entre A_i e A_j . Por exemplo, se A_i libera um slot para A_j , qualquer ação que ocupe esse slot é uma ameaça. A ameaça é resolvida por *promoção* ($A_k \prec A_i$) ou *rebaixamento* ($A_j \prec A_k$). Em um plano totalmente ordenado não há ameaças, porque a ordem já resolve todas.

Linearização e correção. Uma linearização é uma ordem total compatível com $\prec$. O plano de ordem parcial é correto se toda linearização é executável (cada ação satisfaz P1 a P6 no seu momento) e termina no estado meta. A execução manual abaixo testa isso por simulação.

2.3.2 Execução manual: planos do item 3 como ordem parcial

Para cada plano manual do item 3, calculamos os elos causais pelos efeitos de cada ação (colunas Add e Delete do item 3) e testamos todas as ordens de execução das mesmas ações. O script `ordem_parcial.py` automatiza as duas etapas.

Plano	Passos	Elos causais $A_i \xrightarrow{\varphi} A_j$
Sit. 1 (S_{f3})	$A_1 = \text{move}(d, c, 0),$ $A_2 = \text{move}(a, T, 2)$	$A_1 \xrightarrow{\text{livre}(3,1)} A_2$
Sit. 1 (S_{f4})	$A_1 = \text{move}(d, c, 0),$ $A_2 = \text{move}(a, b, 5),$ $A_3 = \text{move}(d, T, 2),$ $A_4 = \text{move}(a, c, 0)$	$A_1 \xrightarrow{\text{livre}(3,1), \text{livre}(5,1)} A_2, A_2 \xrightarrow{\text{livre}(3,0)} A_3,$ $A_3 \xrightarrow{\text{livre}(0,1)} A_4$
Sit. 2 (S_5)	$A_1 = \text{move}(b, T, 2),$ $A_2 = \text{move}(a, b, 2),$ $A_3 = \text{move}(c, d, 4),$ $A_4 = \text{move}(a, c, 4),$ $A_5 = \text{move}(b, c, 5)$	$A_1 \xrightarrow{\text{ocupado}(2,0)} A_2, A_1 \xrightarrow{\text{livre}(1,1)} A_3, A_2 \xrightarrow{\text{livre}(0,1)} A_3,$ $A_3 \xrightarrow{\text{ocupado}(4,1)} A_4, A_3 \xrightarrow{\text{ocupado}(5,1)} A_5,$ $A_4 \xrightarrow{\text{livre}(2,1)} A_5$
Sit. 3 (S_7)	$A_1 = \text{move}(d, c, 0),$ $A_2 = \text{move}(a, b, 5),$ $A_3 = \text{move}(d, T, 2),$ $A_4 = \text{move}(a, c, 0),$ $A_5 = \text{move}(b, c, 1),$ $A_6 = \text{move}(d, T, 3)$	$A_1 \xrightarrow{\text{livre}(3,1), \text{livre}(5,1)} A_2, A_1 \xrightarrow{\text{livre}(3,1), \text{livre}(4,1)} A_6,$ $A_2 \xrightarrow{\text{livre}(3,0)} A_3, A_3 \xrightarrow{\text{livre}(0,1)} A_4, A_3 \xrightarrow{\text{livre}(1,1)} A_5,$ $A_3 \xrightarrow{\text{livre}(2,1)} A_6, A_4 \xrightarrow{\text{livre}(5,1)} A_5, A_5 \xrightarrow{\text{livre}(5,0)} A_6$

Resultado. Nos quatro planos, as restrições de ordem necessárias formam uma cadeia $A_1 \prec A_2 \prec \dots \prec A_k$, e só existe uma ordem de execução válida das mesmas ações. Cada ação libera ou ocupa exatamente o espaço que a seguinte precisa, então não há ações independentes. Com os blocos e as metas dessas situações, o plano mínimo é uma cadeia. Os $A_i \prec A_j$ que não são consecutivos, como $A_1 \prec A_6$ na Situação 3, já decorrem da transitividade da cadeia.

2.3.3 Ordem parcial entre metas

O enunciado pede que a solução considere ordem parcial, $A \xrightarrow{P} B$, e o manual do professor indica como codificá-la: ligar metas por $\varphi_1 \prec_P \varphi_2$. Uma *meta atômica* é $b(p, l)$, isto é, $at(b, p) \wedge lev(b, l)$. Definimos

$$Ach(\varphi, t) \leftrightarrow \exists t' \leq t. \varphi(t') \quad \varphi_1 \prec_P \varphi_2 \iff \forall t. (\varphi_2(t) \rightarrow Ach(\varphi_1, t))$$

Em palavras, a meta φ_2 só pode valer num instante em que φ_1 já valeu antes ou naquele instante. Metas sem relação entre si ficam livres. A relação $\prec_P$ entre metas é irreflexiva e transitiva, como $\prec$ entre passos. Na prática, o plano deve alcançar φ_1 antes de alcançar φ_2 .

Exemplo (Situação 3). As metas são $c(0, 0)$, $d(3, 0)$, $a(0, 1)$ e $b(1, 1)$.

- Com $a(0, 1) \prec_P b(1, 1)$ (colocar a antes de b), o plano mínimo continua com 6 ações, o mesmo do item 3, que já respeita essa ordem: $a(0, 1)$ passa a valer em $t = 4$ e $b(1, 1)$ em $t = 5$.
- Com $b(1, 1) \prec_P a(0, 1)$ (colocar b antes de a), não existe plano com 5 nem com 6 ações. O plano mínimo passa a ter 7: $b(1, 1)$ passa a valer em $t = 5$ e $a(0, 1)$ em $t = 6$. No plano encontrado, o bloco a está sobre b e precisa sair de cima dele antes de b subir. Ele fica temporariamente sobre d e só depois vai para c , o que custa uma ação a mais.

Os tamanhos 5, 6 e 7 vêm da execução do SAT solver para $T = 5, 6, 7$ (seção 6); o plano de 7 ações foi conferido contra as pré-condições P1 a P6.

Plano de 7 ações como ordem parcial. O plano com 7 ações é um caso em que os elos causais deixam ações livres. Para o plano devolvido pelo SAT, $A_1 = move(d, c, 0)$, $A_2 = move(a, b, 5)$, $A_3 = move(d, T, 2)$, $A_4 = move(a, d, 2)$, $A_5 = move(b, c, 1)$, $A_6 = move(a, c, 0)$ e $A_7 = move(d, T, 3)$, as restrições necessárias são

$$A_1 \prec A_2, A_1 \prec A_4, A_2 \prec A_3, A_3 \prec A_5, A_3 \prec A_6, A_4 \prec A_5, A_4 \prec A_6, A_5 \prec A_7, A_6 \prec A_7$$

Existem 3 ordens de execução fisicamente válidas. Só uma delas, $A_1 A_2 A_3 A_4 A_5 A_6 A_7$, executa A_5 antes de A_6 , como a meta $b(1, 1) \prec_P a(0, 1)$ exige. Assim, a ordem parcial entre metas funciona como uma restrição extra, $A_5 \prec A_6$, que escolhe uma entre as ordens permitidas pelos elos causais.

2.4 Execução Manual dos Cenários

Usamos a representação do Item 1 e os efeitos do Item 2. Nível máximo 2 e mesa de 0 a 6; cada bloco é escrito como $b(p, l)$. Em $poss(move(b, y, p), t)$, o nível-alvo é $L = 0$ se $y = T$ e $L = l_y + 1$ se y é bloco. As pré-condições, numeradas, são:

$$(P1) \quad clr(b, t)$$

$$(P2) \quad y \neq b \wedge 0 \leq p \leq 6 - \ell(b) \wedge L \leq 2$$

$$(P3) \quad y = T \vee \exists q. at(y, q, t) \wedge overlap(b, p, y, q)$$

(P4) $\forall s \in [p, p + \ell(b)). \text{free}(s, L, b, t)$

(P5) $\text{stable}(b, p, L, t)$

(P6) $\neg(\text{at}(b, p, t) \wedge \text{lev}(b, L, t))$

Efeitos de t para $t + 1$: ADD $\text{at}(b, p, t + 1)$ e $\text{lev}(b, L, t + 1)$. DELETE da posição anterior p_0 e do nível anterior l_0 , *somente se forem diferentes* de p e L (se $p_0 = p$, o at não é apagado; se $l_0 = L$, o lev não é apagado). Persistência: todo $b' \neq b$ mantém at e lev . Os predicados derivados são recalculados.

2.4.1 Situação 1: $S_0 \rightarrow S_{f3}$

Estado inicial e final conforme o item 1 (S_{f3} é o mais curto dos quatro; os outros levam 4, 8 e 9 movimentos).

Estado inicial: $a(3, 0) \ b(5, 0) \ c(0, 0) \ d(3, 1)$. Estado final: $a(2, 0) \ b(5, 0) \ c(0, 0) \ d(0, 1)$.

t	Ação	P1-P6	Add	Delete	Estado em $t + 1$
0	$\text{move}(d, c, 0)$	✓	$\text{at}(d, 0, 1), \text{lev}(d, 1, 1)$	$\text{at}(d, 3, 1)$	$a(3, 0) \ b(5, 0) \ c(0, 0) \ d(0, 1)$
1	$\text{move}(a, T, 2)$	✓	$\text{at}(a, 2, 2), \text{lev}(a, 0, 2)$	$\text{at}(a, 3, 2)$	$a(2, 0) \ b(5, 0) \ c(0, 0) \ d(0, 1)$

Verificação da *poss.*

- $t = 0$: $\text{move}(d, c, 0)$ ($L = 1$; antes: $d(3, 1)$).
 - (P1) nada cobre os slots $\{3, 4, 5\}$ no nível 2 ✓
 - (P2) $y \neq d, 0 \leq 0 \leq 3, L=1 \leq 2$ ✓
 - (P3) $\text{span}(d, 0) = [0, 3]$ e $\text{span}(c, 0) = [0, 2]$ se sobrepõem ✓
 - (P4) slots $\{0, 1, 2\}$ livres no nível 1 ✓
 - (P5) slots $\{0, 1\}$ apoiados: $2 \geq \lceil 3/2 \rceil = 2$ ✓
 - (P6) $(3, 1) \neq (0, 1)$ ✓
- $t = 1$: $\text{move}(a, T, 2)$ ($L = 0$; antes: $a(3, 0)$).
 - (P1) nada cobre os slots $\{3\}$ no nível 1 ✓
 - (P2) $y \neq a, 0 \leq 2 \leq 5, L=0 \leq 2$ ✓
 - (P3) $y = T$ ✓
 - (P4) slots $\{2\}$ livres no nível 0 ✓
 - (P5) $L = 0$ (apoio na mesa) ✓
 - (P6) $(3, 0) \neq (2, 0)$ ✓

Relações *on* no estado final: $\text{on}(a, T), \text{on}(b, T), \text{on}(c, T), \text{on}(d, a), \text{on}(d, c)$.

2.4.2 Situação 1 (extra): $S_0 \rightarrow S_{f4}$

Plano de 4 movimentos. Coincide com os estados S_0 a S_4 da Situação 3.

Estado inicial: $a(3,0) b(5,0) c(0,0) d(3,1)$. Estado final: $a(0,1) b(5,0) c(0,0) d(2,0)$.

t	Ação	P1-P6	Add	Delete	Estado em $t + 1$
0	$move(d, c, 0)$	✓	$at(d, 0, 1), lev(d, 1, 1)$	$at(d, 3, 1)$	$a(3,0) b(5,0) c(0,0) d(0,1)$
1	$move(a, b, 5)$	✓	$at(a, 5, 2), lev(a, 1, 2)$	$at(a, 3, 2), lev(a, 0, 2)$	$a(5,1) b(5,0) c(0,0) d(0,1)$
2	$move(d, T, 2)$	✓	$at(d, 2, 3), lev(d, 0, 3)$	$at(d, 0, 3), lev(d, 1, 3)$	$a(5,1) b(5,0) c(0,0) d(2,0)$
3	$move(a, c, 0)$	✓	$at(a, 0, 4), lev(a, 1, 4)$	$at(a, 5, 4)$	$a(0,1) b(5,0) c(0,0) d(2,0)$

Verificação da poss.

- $t = 0$: $move(d, c, 0)$ ($L = 1$; antes: $d(3,1)$).
 - (P1) nada cobre os slots $\{3,4,5\}$ no nível 2 ✓
 - (P2) $y \neq d, 0 \leq 0 \leq 3, L=1 \leq 2$ ✓
 - (P3) $span(d,0) = [0,3]$ e $span(c,0) = [0,2]$ se sobrepõem ✓
 - (P4) slots $\{0,1,2\}$ livres no nível 1 ✓
 - (P5) slots $\{0,1\}$ apoiados: $2 \geq \lceil 3/2 \rceil = 2$ ✓
 - (P6) $(3,1) \neq (0,1)$ ✓
- $t = 1$: $move(a, b, 5)$ ($L = 1$; antes: $a(3,0)$).
 - (P1) nada cobre os slots $\{3\}$ no nível 1 ✓
 - (P2) $y \neq a, 0 \leq 5 \leq 5, L=1 \leq 2$ ✓
 - (P3) $span(a,5) = [5,6]$ e $span(b,5) = [5,6]$ se sobrepõem ✓
 - (P4) slots $\{5\}$ livres no nível 1 ✓
 - (P5) slots $\{5\}$ apoiados: $1 \geq \lceil 1/2 \rceil = 1$ ✓
 - (P6) $(3,0) \neq (5,1)$ ✓
- $t = 2$: $move(d, T, 2)$ ($L = 0$; antes: $d(0,1)$).
 - (P1) nada cobre os slots $\{0,1,2\}$ no nível 2 ✓
 - (P2) $y \neq d, 0 \leq 2 \leq 3, L=0 \leq 2$ ✓
 - (P3) $y = T$ ✓
 - (P4) slots $\{2,3,4\}$ livres no nível 0 ✓
 - (P5) $L = 0$ (apoio na mesa) ✓
 - (P6) $(0,1) \neq (2,0)$ ✓
- $t = 3$: $move(a, c, 0)$ ($L = 1$; antes: $a(5,1)$).
 - (P1) nada cobre os slots $\{5\}$ no nível 2 ✓
 - (P2) $y \neq a, 0 \leq 0 \leq 5, L=1 \leq 2$ ✓
 - (P3) $span(a,0) = [0,1]$ e $span(c,0) = [0,2]$ se sobrepõem ✓

- (P4) slots $\{0\}$ livres no nível 1 ✓
- (P5) slots $\{0\}$ apoiados: $1 \geq \lceil 1/2 \rceil = 1$ ✓
- (P6) $(5,1) \neq (0,1)$ ✓

Relações *on* no estado final: $on(a, c), on(b, T), on(c, T), on(d, T)$.

2.4.3 Situação 2: $S_0 \rightarrow S_5$

Plano de 5 movimentos, que passa pelos estados S_1 a S_4 do slide.

Estado inicial: $a(0, 1) b(1, 1) c(0, 0) d(3, 0)$. Estado final: $a(4, 2) b(5, 2) c(4, 1) d(3, 0)$.

t	Ação	P1–P6	Add	Delete	Estado em $t + 1$
0	$move(b, T, 2)$	✓	$at(b, 2, 1), lev(b, 0, 1)$	$at(b, 1, 1), lev(b, 1, 1)$	$a(0, 1) b(2, 0) c(0, 0) d(3, 0)$
1	$move(a, b, 2)$	✓	$at(a, 2, 2), lev(a, 1, 2)$	$at(a, 0, 2)$	$a(2, 1) b(2, 0) c(0, 0) d(3, 0)$
2	$move(c, d, 4)$	✓	$at(c, 4, 3), lev(c, 1, 3)$	$at(c, 0, 3), lev(c, 0, 3)$	$a(2, 1) b(2, 0) c(4, 1) d(3, 0)$
3	$move(a, c, 4)$	✓	$at(a, 4, 4), lev(a, 2, 4)$	$at(a, 2, 4), lev(a, 1, 4)$	$a(4, 2) b(2, 0) c(4, 1) d(3, 0)$
4	$move(b, c, 5)$	✓	$at(b, 5, 5), lev(b, 2, 5)$	$at(b, 2, 5), lev(b, 0, 5)$	$a(4, 2) b(5, 2) c(4, 1) d(3, 0)$

Verificação da poss.

- $t = 0$: $move(b, T, 2)$ ($L = 0$; antes: $b(1, 1)$).
 - (P1) nada cobre os slots $\{1\}$ no nível 2 ✓
 - (P2) $y \neq b, 0 \leq 2 \leq 5, L=0 \leq 2$ ✓
 - (P3) $y = T$ ✓
 - (P4) slots $\{2\}$ livres no nível 0 ✓
 - (P5) $L = 0$ (apoio na mesa) ✓
 - (P6) $(1,1) \neq (2,0)$ ✓
- $t = 1$: $move(a, b, 2)$ ($L = 1$; antes: $a(0, 1)$).
 - (P1) nada cobre os slots $\{0\}$ no nível 2 ✓
 - (P2) $y \neq a, 0 \leq 2 \leq 5, L=1 \leq 2$ ✓
 - (P3) $span(a, 2) = [2, 3]$ e $span(b, 2) = [2, 3]$ se sobrepõem ✓
 - (P4) slots $\{2\}$ livres no nível 1 ✓
 - (P5) slots $\{2\}$ apoiados: $1 \geq \lceil 1/2 \rceil = 1$ ✓
 - (P6) $(0,1) \neq (2,1)$ ✓
- $t = 2$: $move(c, d, 4)$ ($L = 1$; antes: $c(0, 0)$).
 - (P1) nada cobre os slots $\{0,1\}$ no nível 1 ✓
 - (P2) $y \neq c, 0 \leq 4 \leq 4, L=1 \leq 2$ ✓
 - (P3) $span(c, 4) = [4, 6]$ e $span(d, 3) = [3, 6]$ se sobrepõem ✓
 - (P4) slots $\{4,5\}$ livres no nível 1 ✓
 - (P5) slots $\{4,5\}$ apoiados: $2 \geq \lceil 2/2 \rceil = 1$ ✓

- (P6) $(0,0) \neq (4,1)$ ✓
- $t = 3$: $move(a, c, 4)$ ($L = 2$; antes: $a(2, 1)$).
 - (P1) nada cobre os slots $\{2\}$ no nível 2 ✓
 - (P2) $y \neq a$, $0 \leq 4 \leq 5$, $L=2 \leq 2$ ✓
 - (P3) $span(a, 4) = [4, 5]$ e $span(c, 4) = [4, 6]$ se sobrepõem ✓
 - (P4) slots $\{4\}$ livres no nível 2 ✓
 - (P5) slots $\{4\}$ apoiados: $1 \geq \lceil 1/2 \rceil = 1$ ✓
 - (P6) $(2,1) \neq (4,2)$ ✓
- $t = 4$: $move(b, c, 5)$ ($L = 2$; antes: $b(2, 0)$).
 - (P1) nada cobre os slots $\{2\}$ no nível 1 ✓
 - (P2) $y \neq b$, $0 \leq 5 \leq 5$, $L=2 \leq 2$ ✓
 - (P3) $span(b, 5) = [5, 6]$ e $span(c, 4) = [4, 6]$ se sobrepõem ✓
 - (P4) slots $\{5\}$ livres no nível 2 ✓
 - (P5) slots $\{5\}$ apoiados: $1 \geq \lceil 1/2 \rceil = 1$ ✓
 - (P6) $(2,0) \neq (5,2)$ ✓

Relações *on* no estado final: $on(a, c)$, $on(b, c)$, $on(c, d)$, $on(d, T)$.

2.4.4 Situação 3: $S_0 \rightarrow S_7$

Plano de 6 movimentos: $S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow S_3 \rightarrow S_4 \rightarrow S_5 \rightarrow S_7$ (S_6 é igual a S_5 , sem ação entre eles).

Estado inicial: $a(3, 0)$ $b(5, 0)$ $c(0, 0)$ $d(3, 1)$. Estado final: $a(0, 1)$ $b(1, 1)$ $c(0, 0)$ $d(3, 0)$.

t	Ação	P1–P6	Add	Delete	Estado em $t + 1$
0	$move(d, c, 0)$	✓	$at(d, 0, 1)$, $lev(d, 1, 1)$	$at(d, 3, 1)$	$a(3, 0)$ $b(5, 0)$ $c(0, 0)$ $d(0, 1)$
1	$move(a, b, 5)$	✓	$at(a, 5, 2)$, $lev(a, 1, 2)$	$at(a, 3, 2)$, $lev(a, 0, 2)$	$a(5, 1)$ $b(5, 0)$ $c(0, 0)$ $d(0, 1)$
2	$move(d, T, 2)$	✓	$at(d, 2, 3)$, $lev(d, 0, 3)$	$at(d, 0, 3)$, $lev(d, 1, 3)$	$a(5, 1)$ $b(5, 0)$ $c(0, 0)$ $d(2, 0)$
3	$move(a, c, 0)$	✓	$at(a, 0, 4)$, $lev(a, 1, 4)$	$at(a, 5, 4)$	$a(0, 1)$ $b(5, 0)$ $c(0, 0)$ $d(2, 0)$
4	$move(b, c, 1)$	✓	$at(b, 1, 5)$, $lev(b, 1, 5)$	$at(b, 5, 5)$, $lev(b, 0, 5)$	$a(0, 1)$ $b(1, 1)$ $c(0, 0)$ $d(2, 0)$
5	$move(d, T, 3)$	✓	$at(d, 3, 6)$, $lev(d, 0, 6)$	$at(d, 2, 6)$	$a(0, 1)$ $b(1, 1)$ $c(0, 0)$ $d(3, 0)$

Verificação da *poss*.

- $t = 0$: $move(d, c, 0)$ ($L = 1$; antes: $d(3, 1)$).
 - (P1) nada cobre os slots $\{3, 4, 5\}$ no nível 2 ✓
 - (P2) $y \neq d$, $0 \leq 0 \leq 3$, $L=1 \leq 2$ ✓
 - (P3) $span(d, 0) = [0, 3]$ e $span(c, 0) = [0, 2]$ se sobrepõem ✓
 - (P4) slots $\{0, 1, 2\}$ livres no nível 1 ✓
 - (P5) slots $\{0, 1\}$ apoiados: $2 \geq \lceil 3/2 \rceil = 2$ ✓

- (P6) $(3,1) \neq (0,1)$ ✓
- $t = 1$: $move(a, b, 5)$ ($L = 1$; antes: $a(3, 0)$).
 - (P1) nada cobre os slots $\{3\}$ no nível 1 ✓
 - (P2) $y \neq a$, $0 \leq 5 \leq 5$, $L=1 \leq 2$ ✓
 - (P3) $span(a, 5) = [5, 6)$ e $span(b, 5) = [5, 6)$ se sobrepõem ✓
 - (P4) slots $\{5\}$ livres no nível 1 ✓
 - (P5) slots $\{5\}$ apoiados: $1 \geq \lceil 1/2 \rceil = 1$ ✓
 - (P6) $(3,0) \neq (5,1)$ ✓
- $t = 2$: $move(d, T, 2)$ ($L = 0$; antes: $d(0, 1)$).
 - (P1) nada cobre os slots $\{0,1,2\}$ no nível 2 ✓
 - (P2) $y \neq d$, $0 \leq 2 \leq 3$, $L=0 \leq 2$ ✓
 - (P3) $y = T$ ✓
 - (P4) slots $\{2,3,4\}$ livres no nível 0 ✓
 - (P5) $L = 0$ (apoio na mesa) ✓
 - (P6) $(0,1) \neq (2,0)$ ✓
- $t = 3$: $move(a, c, 0)$ ($L = 1$; antes: $a(5, 1)$).
 - (P1) nada cobre os slots $\{5\}$ no nível 2 ✓
 - (P2) $y \neq a$, $0 \leq 0 \leq 5$, $L=1 \leq 2$ ✓
 - (P3) $span(a, 0) = [0, 1)$ e $span(c, 0) = [0, 2)$ se sobrepõem ✓
 - (P4) slots $\{0\}$ livres no nível 1 ✓
 - (P5) slots $\{0\}$ apoiados: $1 \geq \lceil 1/2 \rceil = 1$ ✓
 - (P6) $(5,1) \neq (0,1)$ ✓
- $t = 4$: $move(b, c, 1)$ ($L = 1$; antes: $b(5, 0)$).
 - (P1) nada cobre os slots $\{5\}$ no nível 1 ✓
 - (P2) $y \neq b$, $0 \leq 1 \leq 5$, $L=1 \leq 2$ ✓
 - (P3) $span(b, 1) = [1, 2)$ e $span(c, 0) = [0, 2)$ se sobrepõem ✓
 - (P4) slots $\{1\}$ livres no nível 1 ✓
 - (P5) slots $\{1\}$ apoiados: $1 \geq \lceil 1/2 \rceil = 1$ ✓
 - (P6) $(5,0) \neq (1,1)$ ✓
- $t = 5$: $move(d, T, 3)$ ($L = 0$; antes: $d(2, 0)$).
 - (P1) nada cobre os slots $\{2,3,4\}$ no nível 1 ✓
 - (P2) $y \neq d$, $0 \leq 3 \leq 3$, $L=0 \leq 2$ ✓
 - (P3) $y = T$ ✓
 - (P4) slots $\{3,4,5\}$ livres no nível 0 ✓
 - (P5) $L = 0$ (apoio na mesa) ✓
 - (P6) $(2,0) \neq (3,0)$ ✓

Relações *on* no estado final: $on(a, c)$, $on(b, c)$, $on(c, T)$, $on(d, T)$.

3 Codificação CNF

Para um horizonte de T passos, o problema de planeamento vira um problema de satisfatibilidade: existem os instantes $0, 1, \dots, T$ e as ações ocorrem nos instantes $0, \dots, T-1$. A fórmula é satisfatível se e somente se existe um plano com exatamente T ações que leva do estado inicial ao estado meta. Cada ação $move(b, y, p)$ do item 2 vira uma variável $mv(b, y, p, t)$, e cada predicado da representação em LPO vira uma família de variáveis proposicionais indexadas pelo instante.

3.1 Variáveis proposicionais

Sejam $B = \{a, b, c, d\}$, os pontos $0, \dots, 6$, os slots $0, \dots, 5$ e os níveis $0, 1, 2$.

Variável	Índices	Quantidade
$at(b, p, t)$	$b \in B, p \in [0, 6 - \ell(b)], t \in [0, T]$	$21(T + 1)$
$lev(b, l, t)$	$b \in B, l \in [0, 2], t \in [0, T]$	$12(T + 1)$
$clr(b, t)$	$b \in B, t \in [0, T]$	$4(T + 1)$
$cov(b, s, l, t)$	$b \in B, s \in [0, 5], l \in [0, 2], t \in [0, T]$	$72(T + 1)$
$mv(b, y, p, t)$	$b \in B, y \in (B \setminus \{b\}) \cup \{T\}, p \in [0, 6 - \ell(b)], t \in [0, T - 1]$	$84T$
Total		$193T + 109$

Para $T = 2$, $T = 5$ e $T = 6$ isso dá 495, 1074 e 1267 variáveis.

Os predicados *free*, *stable*, *span* e *overlap* não viram variáveis: o gerador os expande diretamente em cláusulas. A relação *on* também não é codificada (seção 3.5). A variável *cov* foi acrescentada em relação à tabela do manual do professor: ela permite exigir que o apoio de um bloco esteja no nível imediatamente abaixo dele, e não em qualquer nível. Quando há ordem parcial entre metas (grupo 11), entram duas variáveis por meta atômica e por instante, $hold(b, p, l, t)$ e $ach(b, p, l, t)$, isto é, $2(T + 1)$ por meta.

3.2 Grupos de cláusulas

Em todos os grupos, t percorre os instantes em que a regra vale. Os grupos 1 a 6 descrevem os estados, os grupos 7 a 10 descrevem as ações e o grupo 11, opcional, trata a ordem parcial entre metas.

1. **Estado inicial.** Para cada bloco b com $b(p_0, l_0)$ no estado inicial, as cláusulas unitárias $at(b, p_0, 0)$ e $lev(b, l_0, 0)$.
2. **Meta.** Para cada bloco b com $b(p_f, l_f)$ no estado meta, as cláusulas unitárias $at(b, p_f, T)$ e $lev(b, l_f, T)$.
3. **Unicidade de posição e de nível** (leis U1 e U2). Para cada b e t :

$$\bigvee_p at(b, p, t) \quad \neg at(b, p_1, t) \vee \neg at(b, p_2, t) \quad (p_1 < p_2)$$

e o análogo para lev com $l \in \{0, 1, 2\}$.

4. **Definição de cov.** Para cada b, s, l e t , com $P_s = \{p : p \leq s < p + \ell(b)\}$:

$$\begin{aligned} cov(b, s, l, t) &\rightarrow lev(b, l, t) \\ cov(b, s, l, t) &\rightarrow \bigvee_{p \in P_s} at(b, p, t) \\ at(b, p, t) \wedge lev(b, l, t) &\rightarrow cov(b, s, l, t) \quad (p \in P_s) \end{aligned}$$

5. **Exclusão horizontal** (lei E). Para blocos distintos b, b' , posições p, q com $overlap(b, p, b', q)$ e todo nível l :

$$\neg at(b, p, t) \vee \neg lev(b, l, t) \vee \neg at(b', q, t) \vee \neg lev(b', l, t)$$

6. **Clear.** Se $clr(b, t)$ vale, nenhum outro bloco z está no nível seguinte sobreposto a b . Para $overlap(b, p, z, q)$ e $l < 2$:

$$\neg clr(b, t) \vee \neg at(b, p, t) \vee \neg lev(b, l, t) \vee \neg at(z, q, t) \vee \neg lev(z, l + 1, t)$$

O gerador escreve também cláusulas na direção contrária, mas elas não são necessárias para a correção: clr só aparece como pré-condição de $move$, e o solver pode escolher $clr(b, t)$ verdadeiro sempre que nada está sobre b .

7. **Ação única por instante.** Para cada $t \in [0, T - 1]$, pelo menos uma variável $mv(\cdot, t)$ é verdadeira, e para cada par de ações distintas $\neg mv_1 \vee \neg mv_2$.

8. **Pré-condições de move.** Para cada $mv(b, y, p, t)$, com $L = 0$ se $y = T$ e $L = l_y + 1$ caso contrário:

- (P1) $mv(b, y, p, t) \rightarrow clr(b, t)$.
- (P2) $y \neq b$ e $0 \leq p \leq 6 - \ell(b)$ já valem pelo domínio das variáveis. O limite $L \leq 2$ vira $mv(b, y, p, t) \rightarrow \neg lev(y, 2, t)$.
- (P3) Se $y \neq T$, para cada posição q de y sem sobreposição: $\neg mv(b, y, p, t) \vee \neg lev(y, l_y, t) \vee \neg at(y, q, t)$.
- (P4) Para cada outro bloco z e posição q com $overlap(b, p, z, q)$: $\neg mv(b, y, p, t) \vee \neg lev(y, l_y, t) \vee \neg at(z, q, t) \vee \neg lev(z, L, t)$. Para $y = T$, o mesmo com $L = 0$ e sem a parcela de y .
- (P5) Estabilidade, descrita a seguir.
- (P6) $\neg mv(b, y, p, t) \vee \neg at(b, p, t) \vee \neg lev(y, l_y, t) \vee \neg lev(b, L, t)$, o que evita ações que não mudam o estado.

9. **Efeitos de move.** $mv(b, y, p, t) \rightarrow at(b, p, t + 1)$, e $mv(b, y, p, t) \wedge lev(y, l_y, t) \rightarrow lev(b, l_y + 1, t + 1)$. Para $y = T$, $mv(b, T, p, t) \rightarrow lev(b, 0, t + 1)$.

10. **Frame axioms.** Para $b' \neq b$, $mv(b, y, p, t) \wedge at(b', p', t) \rightarrow at(b', p', t + 1)$ e $mv(b, y, p, t) \wedge lev(b', l', t) \rightarrow lev(b', l', t + 1)$.

11. **Ordem parcial entre metas (opcional).** Para cada relação $\varphi_1 \prec_P \varphi_2$, com metas atômicas $\varphi = b(p, l)$, cláusulas que dizem que φ_2 só vale em t se φ_1 já valeu em algum $t' \leq t$:

$$\begin{aligned} hold(\varphi, t) &\rightarrow at(b, p, t) & hold(\varphi, t) &\rightarrow lev(b, l, t) & at(b, p, t) \wedge lev(b, l, t) &\rightarrow hold(\varphi, t) \\ ach(\varphi, 0) &\rightarrow hold(\varphi, 0) & ach(\varphi, t) &\rightarrow ach(\varphi, t - 1) \vee hold(\varphi, t) & (t \geq 1) \\ hold(\varphi_2, t) &\rightarrow ach(\varphi_1, t) & (t \in [0, T]) \end{aligned}$$

Basta a direção “*ach* implica que já valeu”: como *ach* só aparece positivo na última cláusula, o solver só consegue torná-lo verdadeiro se φ_1 de fato valeu antes. É o que o manual do professor descreve como $\neg\varphi_2(t) \vee \varphi_1(t')$ para algum $t' \leq t$.

3.3 Estabilidade em cláusulas

A condição (P5) exige que pelo menos $k = \lceil \ell(b)/2 \rceil$ dos $n = \ell(b)$ slots sob b estejam apoiados no nível l_y . “Pelo menos k de n slots apoiados” equivale a “todo subconjunto de $n - k + 1$ slots tem algum slot apoiado”. Então, para cada subconjunto S de $n - k + 1$ slots de $\text{span}(b, p)$:

$$\neg mv(b, y, p, t) \vee \neg lev(y, l_y, t) \vee \bigvee_{s \in S} \bigvee_{z \neq b} cov(z, s, l_y, t)$$

Para $b = d$ ($n = 3, k = 2$) são três cláusulas, uma para cada par de slots. Para a, b e c ($k = 1$ e $n - k + 1 = n$) é uma cláusula só, e ela exige apoio em algum dos slots sob o bloco. Quando o destino é a mesa ($y = T$), não há cláusula de estabilidade.

3.4 Como ADD e DELETE aparecem na codificação

O ADD do item 2 são as cláusulas de efeito do grupo 9. Não existe cláusula de DELETE explícita: a posição e o nível anteriores deixam de valer porque as cláusulas de unicidade (grupo 3) proíbem duas posições ou dois níveis para o mesmo bloco no mesmo instante. Isso é coerente com a regra do item 2 de só apagar o que muda: se $p_0 = p$, o *at* novo e o antigo são o mesmo átomo e continuam verdadeiros.

3.5 A relação *on* é derivada

Não criamos variáveis para $on(b, y, t)$. A relação é reconstruída depois, a partir do modelo do SAT solver:

$$on(b, y, t) \leftrightarrow \exists s, l. cov(b, s, l, t) \wedge cov(y, s, l - 1, t) \quad on(b, T, t) \leftrightarrow lev(b, 0, t)$$

Isso mantém o número de variáveis menor, e o script `interpretar.py` faz a reconstrução ao apresentar o plano.

3.6 Horizonte e plano mínimo

Como cada instante tem exatamente uma ação, a fórmula para T é satisfatível quando existe plano de exatamente T ações. Para achar o plano mais curto, executa-se o SAT solver com $T = 1, 2, 3, \dots$ e para-se no primeiro T satisfatível. Esse T é o comprimento mínimo do plano, porque os valores menores deram insatisfatível.

4 Exemplos dos 3 Cenários Codificados para CNF

Cada cenário vira um arquivo `.cnf` (as cláusulas) e um `.map` (o nome de cada variável). O estado inicial e o estado meta entram como cláusulas unitárias nos instantes 0 e T , e as demais cláusulas são as do grupo 3 em diante da seção anterior. O horizonte T usado em cada caso é o comprimento do plano mínimo.

Cenário	Estado inicial	Estado meta	T	Variáveis	Clá
Situação 1 ($S_0 \rightarrow S_{f3}$)	$c(0,0) a(3,0) b(5,0) d(3,1)$	$c(0,0) a(2,0) b(5,0) d(0,1)$	2	495	
Situação 2 ($S_0 \rightarrow S_5$)	$c(0,0) a(0,1) b(1,1) d(3,0)$	$c(4,1) a(4,2) b(5,2) d(3,0)$	5	1 074	
Situação 3 ($S_0 \rightarrow S_7$)	$c(0,0) a(3,0) b(5,0) d(3,1)$	$c(0,0) a(0,1) b(1,1) d(3,0)$	6	1 267	

4.1 Situação 1

Estado inicial S_0 : $c(0,0) a(3,0) b(5,0) d(3,1)$, com d em ponte sobre a e b . Meta S_{f3} : $c(0,0) a(2,0) b(5,0) d(0,1)$. Horizonte $T = 2$. O arquivo `trab01_blocos2SAT_situacao1.map` associa, por exemplo, 480 a `move(d,c,0,0)` e 369 a `move(a,T,2,1)`. As 16 cláusulas unitárias são as do estado inicial (8, em $t = 0$) e as da meta (8, em $t = 2$):

```
t=0: at(a,3,0) lev(a,0,0) at(b,5,0) lev(b,0,0) at(c,0,0) lev(c,0,0) at(d,3,0)
      lev(d,1,0)
t=2: at(a,2,2) lev(a,0,2) at(b,5,2) lev(b,0,2) at(c,0,2) lev(c,0,2) at(d,0,2)
      lev(d,1,2)
```

As outras três metas da Situação 1 (S_{f1} , S_{f2} e S_{f4}) são geradas com o terceiro argumento, por exemplo `python3 gerar_cnf.py 1 4 f4`.

4.2 Situação 2

Estado inicial S_0 : $c(0,0) a(0,1) b(1,1) d(3,0)$, com a e b lado a lado sobre c . Meta S_5 : $c(4,1) a(4,2) b(5,2) d(3,0)$, com c sobre d e a e b lado a lado sobre c . Horizonte $T = 5$.

4.3 Situação 3

Estado inicial S_0 : o mesmo da Situação 1. Meta S_7 : $c(0,0) a(0,1) b(1,1) d(3,0)$, com a e b lado a lado sobre c e d na mesa. Horizonte $T = 6$. O estado S_6 do slide é igual a S_5 e não gera ação.

4.4 Cenário com ordem parcial entre metas

O grupo (ORD) acrescenta variáveis e cláusulas só quando há ordem entre metas. Para a Situação 3 com $b(1,1) \prec_P a(0,1)$ e $T = 7$, o CNF tem 1 492 variáveis e 59 795 cláusulas, contra 1 267 e 51 308 do caso sem ordem com $T = 6$. O comando é `python3 gerar_cnf.py 3 7 -ordem='b(1,1)<a(0,1)'`.

5 Mapeamento: Descrição Formal $\rightarrow$ Código

A tabela liga cada elemento da descrição formal (itens 1 e 2) ao trecho correspondente de `gerar_cnf.py`. Os blocos do código são identificados pelos comentários do próprio arquivo.

Regra formal	Código Python (<code>gerar_cnf.py</code>)
Blocos e comprimentos $\ell(b)$	<code>BLOCKS = {"a":1, "b":1, "c":2, "d":3}</code>

Regra formal	Código Python (<i>gerar_cnf.py</i>)
Pontos do eixo e nível máximo	<code>MAX_POINT = 6, MAX_LEVEL = 2</code>
Posições válidas $p \in [0, 6 - \ell(b)]$	<code>positions(b)</code>
$overlap(b, p, y, q)$	<code>overlap(b, p, y, q)</code>
at, lev, clr, cov, mv	dicionários <code>at, lev, clr, cov, mv</code> , criados com <code>C.new_var(...)</code>
Estado inicial e meta (grupos 1 e 2)	bloco # Estado inicial e estado meta
Unicidade de posição e nível, U1 e U2 (grupo 3)	bloco # (A,B)
Definição de cov (grupo 4)	bloco # (COV)
Exclusão horizontal, lei E (grupo 5)	bloco # (C)
clr (grupo 6)	bloco # (E)
Ação única por instante (grupo 7)	início do laço # Uma ação por instante...
P1: $clr(b, t)$	<code>C.add(-v, clr[b, t])</code>
P2: $L \leq 2$	<code>C.add(-v, -lev[y, MAX_LEVEL, t])</code>
P3: sobreposição com o apoio	laço <code>for q in positions(y): if not overlap(...)</code>
P4: $free$ nos slots de destino	trechos # Destino na mesa... e # Não coincidir horizontalmente...
P5: $stable$	bloco # Estabilidade..., com <code>itertools.combinations(slots, r)</code> e <code>cov[z, s, ly, t]</code>
P6: $\neg(at \wedge lev)$	bloco # Evita no-op...
Efeitos ADD de $move$ (grupo 9)	<code>C.add(-v, at[b, p, t+1])</code> e <code>C.add(-v, lev[b, ..., t+1])</code>
DELETE	não há código: vem da unicidade, bloco # (A,B)
Persistência (grupo 10)	bloco # Frame axioms...
Ordem parcial entre metas (grupo 11)	bloco # (ORD), com as variáveis <code>ach</code> e <code>hold</code> ; opção <code>-ordem</code> lida por <code>ler_ordem(...)</code>
Metas da Situação 1 (S_{f1} a S_{f4})	dicionário <code>METAS_SITUACA01</code>
Plano mínimo (menor T satisfatível)	<code>buscar_plano.py</code> , laço sobre $T = 1, 2, \dots$
Verificação independente de P1 a P6	<code>interpretar.py</code> , função <code>validar_plano</code>
Elos causais e ordem parcial do plano	<code>ordem_parcial.py</code>
$on(b, y, t)$ derivado	<code>interpretar.py</code> , função <code>derivar_on</code>

5.1 Diferenças em relação ao manual do professor

- O manual exige $clr(y)$ na pré-condição de $move$. Aqui a exigência foi trocada por $free$ por slot (P4), como no item 1. Sem isso, as transições $S_4 \rightarrow S_5$ das Situações

2 e 3 seriam impossíveis.

- Foi acrescentada a variável *cov*, para que o apoio da estabilidade seja contado no nível certo.
- O resultado do SAT solver é lido com o arquivo `.map`, que associa cada número de variável ao seu símbolo, como *mv*(*d*, *c*, 0, 0).

6 Execução Passo a Passo

Requisitos: Python 3 e o minisat no PATH. Os arquivos ficam todos na mesma pasta.

6.1 Uma situação, passo a passo

1. **Gerar o CNF e o mapa.** O primeiro argumento é a situação e o segundo é o horizonte *T*.

```
python3 gerar_cnf.py 3 6
```

Saída: Situação 3 (T=6): 1267 variáveis, 51308 cláusulas. Arquivos gerados: `trab01_blocos2SAT_situacao3.cnf` e `trab01_blocos2SAT_situacao3.map`.

2. **Executar o minisat.**

```
minisat trab01_blocos2SAT_situacao3.cnf resultado3.txt
```

O minisat escreve em `resultado3.txt` a palavra SAT seguida da lista de literais, ou UNSAT.

3. **Interpretar o resultado.** O script usa o `.map` correspondente.

```
python3 interpretar.py resultado3.txt --verbose
```

6.2 Plano mínimo automático

O comando `python3 buscar_plano.py 3` repete os passos acima com $T = 1, 2, 3, \dots$ e para no primeiro SATISFIABLE. Esse *T* é o comprimento mínimo do plano, porque os valores menores deram UNSATISFIABLE. O script grava o `resultado3.txt` do último *T* e chama o `interpretar.py`.

6.3 Horizontes testados

O resultado esperado, que é uma propriedade do problema e não do solver, é:

Cenário	UNSAT para	SAT em	Plano mínimo
Situação 1 (S_{f3})	$T = 1$	$T = 2$	2 ações
Situação 1 (S_{f4})	$T = 1, 2, 3$	$T = 4$	4 ações
Situação 2 (S_5)	$T = 1, \dots, 4$	$T = 5$	5 ações
Situação 3 (S_7)	$T = 1, \dots, 5$	$T = 6$	6 ações

Para S_{f1} e S_{f2} da Situação 1, a busca em largura sobre os 2032 estados alcançáveis dá planos mínimos de 8 e 9 ações. Elas são geradas com `python3 buscar_plano.py 1 f1` e `python3 buscar_plano.py 1 f2`.

6.4 Com ordem parcial entre metas

A ordem entre metas entra com a opção `-ordem`. Na Situação 3, exigir $b(1,1)$ antes de $a(0,1)$ dá $T = 5$ e $T = 6$ UNSAT e $T = 7$ SAT:

```
python3 buscar_plano.py 3 --tmax 8 --ordem='b(1,1)<a(0,1)'
```

Os arquivos desse cenário têm o sufixo `_ordem`. Já exigir $a(0,1)$ antes de $b(1,1)$ mantém o plano de 6 ações.

6.5 Arquivos entregues

Para cada cenário: `trab01_blocos2SAT_situacaoN.cnf`, `trab01_blocos2SAT_situacaoN.map` e `resultadoN.txt`, com $N = 1, 2, 3$. Os `resultadoN.txt` são a saída real do minisat para o menor T satisfável.

7 Interpretação da Saída do SAT Solver

7.1 O minisat não conhece nomes simbólicos

O minisat é um resolvidor puramente booleano. Ele devolve SAT (ou UNSAT) e uma lista de inteiros, onde um número positivo é uma variável verdadeira e um negativo é uma variável falsa. Toda a tradução de número para símbolo vem do arquivo `.map`, que o `gerar_cnf.py` escreve junto com o CNF. Sem o `.map`, a saída é só uma lista de inteiros sem significado para o domínio.

7.2 Da saída numérica ao plano em português

O script `interpretar.py` executa estes passos:

1. **Leitura do mapa:** associa cada número ao seu símbolo, por exemplo $480 \rightarrow \text{move}(d, c, 0, 0)$.
2. **Filtragem:** mantém os literais positivos. As variáveis `move` formam o plano, e as variáveis `at` e `lev` dão o estado em cada instante.
3. **Ordenação temporal:** agrupa as ações por t e traduz cada uma para uma frase em português.
4. **Derivação de *on*:** no estado final, $on(b, T)$ se b está no nível 0. Caso contrário, $on(b, y)$ para todo bloco y no nível $l - 1$ cujo *span* se sobrepõe ao de b . Isso lista, por exemplo, d sobre a e b em ponte, ou a sobre c .
5. **Verificação independente:** confere cada ação contra as pré-condições P1 a P6 do item 1 e confere se o estado $t + 1$ é exatamente o efeito da ação. Essa verificação não usa o CNF, então detecta erro da codificação.

7.3 Exemplo: Situação 3

Saída do `interpretar.py` para um plano mínimo da Situação 3 (6 ações). O minisat pode devolver outro plano com o mesmo número de ações; o que deve coincidir é o tamanho e a validade.

PLANO ENCONTRADO (6 ações):

1. t=0: mover bloco 'd' para CIMA de 'c' em p=0
2. t=1: mover bloco 'a' para CIMA de 'b' em p=5
3. t=2: mover bloco 'd' para a MESA em p=2
4. t=3: mover bloco 'a' para CIMA de 'c' em p=0
5. t=4: mover bloco 'b' para CIMA de 'c' em p=1
6. t=5: mover bloco 'd' para a MESA em p=3

ESTADO FINAL (t=6):

- a: ponto inicial p=0, nível l=1
- b: ponto inicial p=1, nível l=1
- c: ponto inicial p=0, nível l=0
- d: ponto inicial p=3, nível l=0

RELACOES 'on' em t=6:

- a esta sobre: c
- b esta sobre: c
- c esta sobre: MESA
- d esta sobre: MESA

VERIFICAÇÃO INDEPENDENTE (pré-condições P1..P6 do Item 1): plano válido

7.4 Comparação com os planos manuais

A comparação é pelo *tamanho* e pela *validade*, e não pela sequência idêntica, porque um mesmo problema pode ter vários planos mínimos. Por exemplo, na Situação 2 o plano manual leva *b* para a mesa na primeira ação, e um plano igualmente válido de 5 ações começa com *a* sobre *d*.

Cenário	Ações no plano manual	T mínimo no SAT	Mesmo tamanho
Situação 1 (S_{f3})	2	2	sim
Situação 1 (S_{f4})	4	4	sim
Situação 2 (S_5)	5	5	sim
Situação 3 (S_7)	6	6	sim

Se o SAT desse UNSAT para um T menor do que o plano manual, o resultado seria o esperado, porque o plano manual é um limite superior do menor T . Se desse SAT com T menor e o `interpretar.py` confirmasse o plano como válido, o plano manual não seria mínimo. Se desse SAT e o plano fosse inválido, haveria erro na codificação. Nos quatro cenários da tabela, o menor T do SAT coincide com o do plano manual e o plano devolvido passa na verificação independente.

Exemplo do manual do professor. O plano de exemplo da seção 6 do manual (*d* sobre *c*; *a* para a mesa em $p = 4$; *d* para a mesa em $p = 2$; *a* sobre *c*) não é válido: no

terceiro passo, d em $p = 2$ cobre os slots 2, 3 e 4, e o slot 4 já está ocupado por a no nível 0. Por isso ele não foi usado como gabarito.

7.5 Com ordem parcial entre metas

Com a opção `-ordem`, o `interpretar.py` também confere se a ordem foi respeitada: para cada $\varphi_1 \prec_P \varphi_2$, em todo instante em que φ_2 vale, φ_1 já valeu. O `ordem_parcial.py` imprime, para o plano encontrado, os elos causais $A_i \xrightarrow{\varphi} A_j$ e as restrições de ordem necessárias, como na seção sobre ordem parcial.